

**TEKNOFEST
HAVACILIK, UZAY ve TEKNOLOJİ
FESTİVALİ**

ULAŞIMDA YAPAY ZEKA YARIŞMASI

KRİTİK TASARIM RAPORU



FEZA AI TAKIMI

TAKIM ID:

464371

İÇİNDEKİLER

1. TAKIM ŞEMASI	1
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ	1
3. ALGORİTMALAR VE SİSTEM MİMARİSİ.....	2
3.1 Veri Setleri	2
3.2 Algoritmalar	3
4. ÖZGÜNLÜK	6
5. SONUÇLAR VE İNCELEME	8
6. KAYNAKÇA	10



1. Takım Şeması

FEZA AI takımı, üç üyesi bilgisayar mühendisliği, bir üyesi elektronik mühendisliği ve bir üyesi de elektrik ve elektronik mühendisliği okumakta olan toplamda 5 mühendislik lisans öğrencisinden oluşmaktadır.

Takımımız ön tasarım raporunda da belirtildiği üzere “bilgisayarlı görü” ve “makine öğrenmesi” birimi olmak üzere iki alt birimden oluşmaktadır.

Bilgisayarlı görü birimi, nesne tespit algoritmaları, nesne takibi, tespit edilen nesnenin biçim özelliklerinin kaydedilmesi ve raporlanması, nesne tespit algoritmalarının verim ve performans analizleri gibi konularda çalışmalar yapmaktadır.

Makine öğrenmesi birimi ise, istenilen niteliklerde veri tabanı bulunması, verilerin düzenlenmesi, makine öğrenmesi algoritmaları, veriler ile algoritmanın eğitilmesi gibi alanlarda çalışmalar yapmaktadır.

Takım üyelerinin teknik görevlerinin yanı sıra bir adet de teknik olmayan görevleri bulunmaktadır. Üyelerimizin görev dağılımı aşağıdaki tablodaki gibidir (Resim 1.1).

TAKIM ÜYESİ		Teknik Görev	Teknik Olmayan Görev	
1	Kaptan	Proje Yönetimi ve Test Sorumlusu Projenin geliştirileceği ortamların (Github vb.) kurulması, projenin çalıştırılacağı sanal makinelerin kurulması ve düzenli aralıklarla projenin test edilip raporlanması	Kaptanlık	Makine Öğrenmesi Birimi
2	Üye 1	Makine Öğrenmesi Algoritmaları Sorumlusu Makine öğrenmesi algoritmalarının karşılaştırılarak proje için en uygun olanın belirlenmesi ve gerekli sistemlerin kurulması	Sosyal Medya Yönetimi	
3	Üye 2	Veri ve Veri Tabanı Sorumlusu Proje için istenilen niteliklerde veri tabanlarının bulunması ve düzenlenmesi	Sponsorluk Görüşmeleri	
4	Üye 3	Bilgisayarlı Görü Algoritmaları Sorumlusu Bilgisayarlı görü algoritmalarının verim ve zaman bazlı olarak karşılaştırılması ardından karar verilen algoritma için gerekli düzenlenmelerin yapılması	Akademik Danışmanlar ile İletişim	Bilgisayarlı Görü Birimi
5	Üye 4	Nesne Tespit Sistemi ve Yapay Veri Sorumlusu Nesne tespiti için gerekli olan görsellerin yapayların üretimi ve tespit edilen nesnelerin bilgilerinin aktarılması	Finans Sorumlusu	

Resim 1.1

2. Proje Mevcut Durum Değerlendirmesi

Projemizin geliştirilme aşamasında akademik danışmanlarımızdan gelen geri bildirimler doğrultusunda, Rohith Gandhi'nin 2018 yılında Medium platformu üzerinden Towards Data Science yayının altında yayınladığı “R-CNN, Fast R-CNN, Faster R-CNN, YOLO — Object Detection Algorithms” makalesi temel alınarak bir kez daha nesne tespit algoritmaları karşılaştırılması yapılmıştır.

Danışmanlarımızın önerileri ve makaledeki teknik değerlendirmeler incelendiğinde projemiz için en iyi algoritma seçiminin ön tasarım raporundan farkı olarak “YOLO” yerine “Fast R-CNN” algoritması olduğuna karar kılınmıştır.

Lakin, projenin geliştirilme aşamasına geçildiğinde üyelerin teknik bilgi birikimin lisans düzeyinde olması, üyelerin birçoğunun YOLO algoritması ile en az bir proje geliştirmesi, literatür taraması yapıldığında örnek alınabilecek projelerin birçoğunda YOLO algoritmasının tercih edilmesi gibi avantajlar ve dezavantajlar bir kez daha tartışılıp kısıtlı zaman imkanlarını da göz önüne alarak tekrardan YOLO algoritmasının kullanılmasına karar kılınmıştır.

Ardından raporun “4. Özgünlük” kısmında daha detaylı bir şekilde anlatılan yapay görseller oluşturulmuştur. Oluşturulan yapay görseller YOLO algoritması ile çalıştırılmış ve istenilen nitelikte sonuçlar elde edilmiştir. Projenin mevcut durumunda ise gerçek görüntülerden oluşan veri tabanları ve

yapay görseller üzerinde çalışılmaya devam edilmektedir. Makine öğrenmesi kütüphaneleri de kullanılarak hedeflenen “%70 ve üzeri” nesne tespit oranına ulaşılmaya çalışılmaktadır.

3. Algoritmalar ve Sistem Mimarisi

3.1 Veri Setleri

Makine öğrenmesi birimimiz veri setleri araştırması yaparken teknik şartnameyi ve algoritmayı eğitmemiz için verilen örnek videoyu temel almıştır. Yüksek oranda nesne tespiti yapabilen ve farklı durumlara olabildiğince hazır durumda olan bir sistem geliştirmek için aşağıdaki 4 ana nitelik üzerinden ilerlenmiştir.

1. Bulunacak insansız hava araçları görüntüleri örnek videodaki gibi yağmurlu, karlı, sisli hava koşullarını içermelidir.
2. Sistemin oluşabilecek tüm farklı durumlara hazırlıklı olabilmesi için görüntüler gün içinden farklı zamanlardan çekilmiş olmalıdır.
3. Bulunacak veriler örnek videodaki gibi otoyol, meskûn mahal, farklı ulaşım araçları, farklı durumlarda bulunan insanlar ve nesne tespiti yapılan alanda bulunma zamanı az olan nesneler gibi farklı niteliklerde nesneler içermelidir.
4. Görüntüler örnek videodaki gibi 60° – 90° açı aralığından çekilmiş olup kuş bakışı görüntüler içermelidir.

Veri Setlerimizden birkaç örnek resim, Teknofest tarafından bizlere gönderilen örnek video ile karşılaştırılmalı olarak aşağıda gösterilmiştir. (Resim 3.1.1)

Veri Setlerimizden Örnek Görüntüler



Teknofest Tarafından Bizlere Verilen Videodan Örnek Görüntüler

Resim 3.1.1

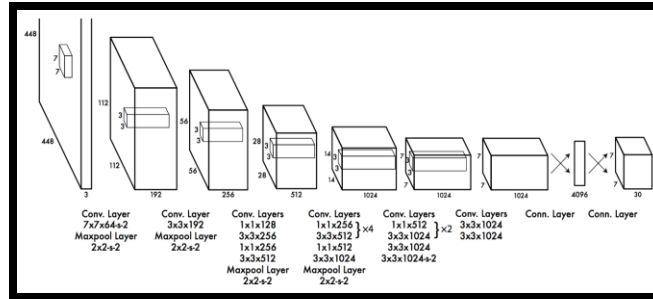
Veri setlerimizin linkleri aşağıdaki drive linkindedir:

<https://drive.google.com/file/d/1A17Xc3pgdl7bLr4dfEhwmMlplcO9vsbp/view?usp=sharing>

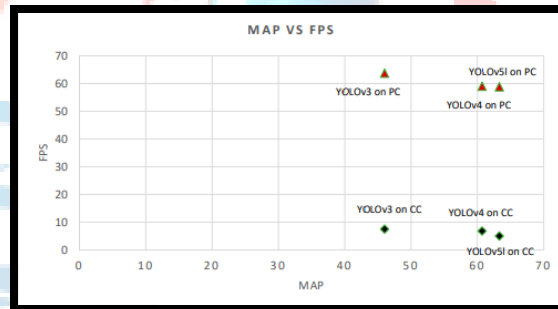
3.2 Algorithmen

Proje mevcut durum değerlendirmesi kısmında belirtildiği üzere takımımız tekrardan YOLO (You Only Look Once) algoritmasını kullanmaya devam etmektedir.

YOLO, Resim 3.2.1’de görüldüğü üzere konvolüsyonel sinir ağları kullanarak nesne tespiti yapan bir algoritmadır.

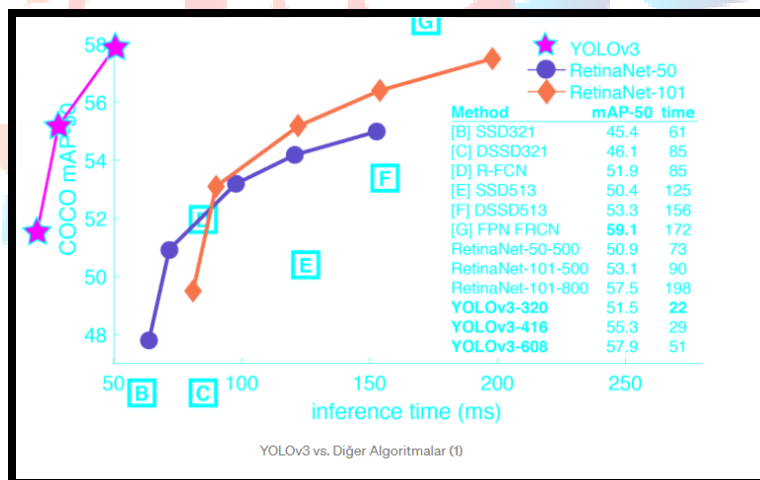


Resim 3.2.1



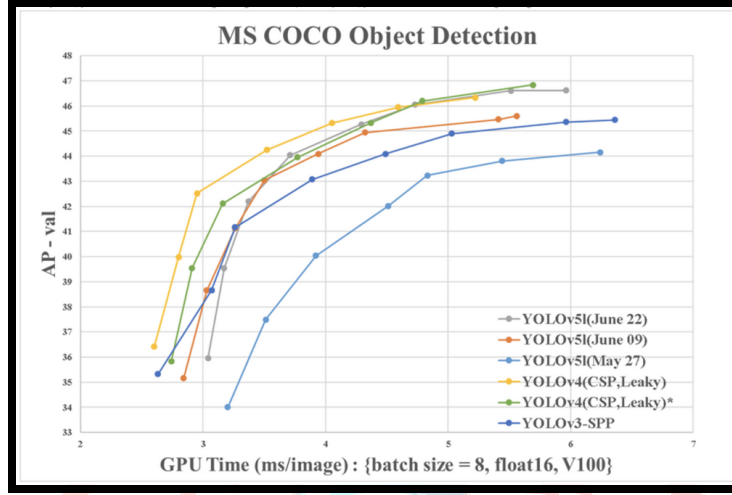
Resim 3.2.2

Resim 3.2.2'de de görüleceği üzere 40 – 80 FPS (Frames per Second) işleyebilir. Bundan dolayı diğer algoritmalara göre oldukça hızlı tahmin yapıp videonun gerçek zamanlı olarak işlenmesini sağlayabilmektedir.



Resim 3.2.3

Resim 3.2.3'de YOLO V3 ve diğer nesne tespit algoritmalarının COCO veri setinde karşılaştırılmasını gösterilmiştir. Grafikten de anlaşılacağı üzere YOLO diğer algoritmalarla karşı “süre” ve “doğruluk payı” açısından oldukça avantajlıdır.



Resim 3.2.4

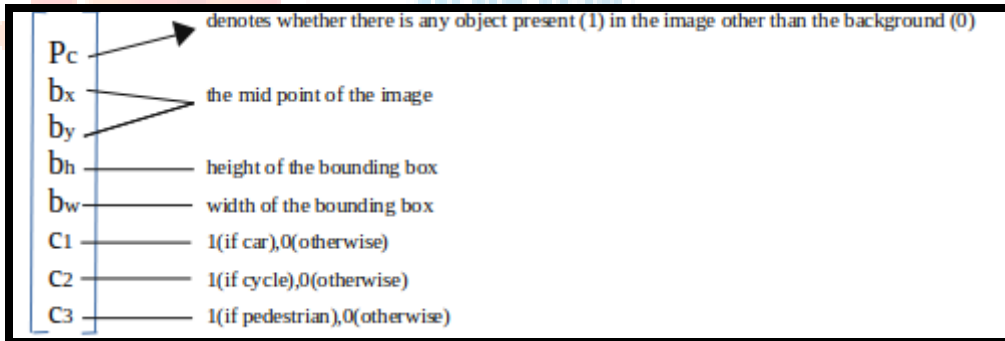
Resim 3.2.4 incelendiğinde ise YOLO V4 ve YOLO V5 algoritmaları diğer YOLO versiyonlarına göre daha gelişmiştir. Takımımız, danıştığımız akademisyenlerden de gelen geri bildirimleri de göz önüne alarak kullanım kolaylığından dolayı YOLO V5 algoritmasını projemizde kullanmaya karar vermiştir.

Algoritma Çalışma Sistemi ve Sistem Mimarisi

YOLO konvolüsyonel sinir ağları (CNN) ile bir girdi görüntüsü alıp görüntüdeki nesneleri, görünüşleri birbirinden ayırabilen bunları tanımlayabilen bir derin öğrenme algoritmasıdır.

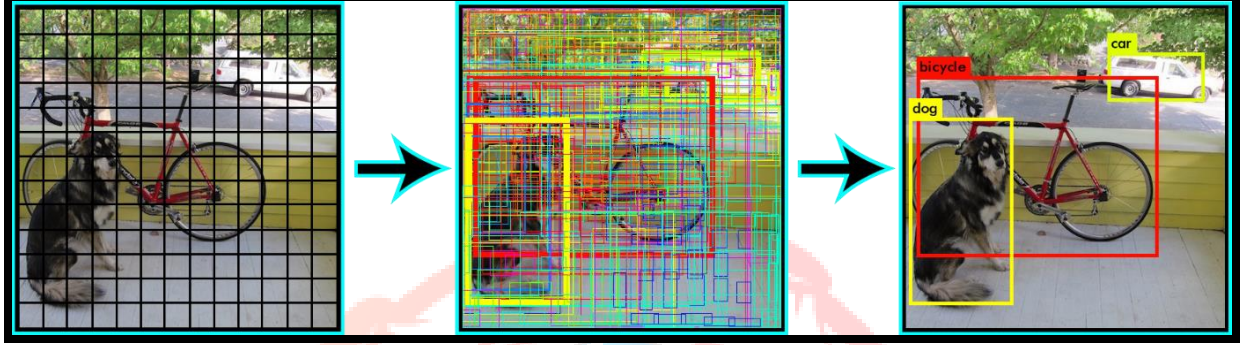
YOLO girdi olarak alınan bir resmi veya videoyu bölgelere ayırır (grid) ve bu SxS şeklinde parçalar olabilir.

Daha sonra bölgelere ayrılan kare nöron ağıyla tekrar boyutlandırılır. Bu boyutlandırma işlemi şu şekilde gerçekleşir; parçalara ayrılan her kutucukta obje kontrolü yapılır. Eğer objenin merkezi bu kutucukta ise bu grid objeden sorumlu olacak şekilde tahmini hedef vektör matrisi çıkartılır (Resim 3.2.5). Bu vektör matrislerinin içinde nesnenin merkez koordinatları, kutucuğun yükseklik- genişlik değerleri, Objelerin tanımlanıp tanımlanmadığı bilgisi bulunur ve güven skoru belirlenir.



Resim 3.2.5 Target Label Matrix

En sonunda görüntüye **non-max suppression** uygulanır. Bu uygulama şu şekilde gerçekleşir. Nesneyi kapsayan birden fazla sınırlayıcı kutucuk çizilebilir. Bu kutucuklar arasında güven skoru düşük olanlar temizlenir. Bu şekilde en yüksek skor elde edilir (Resim 3.2.6).



Resim 3.2.6 non-max suppression

Özellikle belirtmemiz gerekir ki YOLOv5'i seçme nedenlerimizden biri PyTorch Hub'dır. PyTorch Hub, daha önce eğitilmiş modeli, PyTorch kütüphanesinin metotlarını kullanarak model parametrelerine müdahale edebilme imkânı sunmaktadır. Bu parametreler arasında nesne sınıfları gibi parametreler mevcuttur. Ayrıca modelimizin sonuçlarını JSON olarak aktarabilmemize yarayan metoda da sahiptir. Yarışma koşulları gereği tespit edilen nesnenin tüm detaylarını; x-y eksen koordinat bilgileri, nesnenin sınıfı gibi bilgileri direkt olarak sunma potansiyeli de bulunmaktadır.

Böylece modelimizden elde ettiğimiz çıkışı direkt olarak yarışma koşullarınca aktarabilmemiz mümkündür.

Kullanacağımız YOLOv5 modeli, diğer YOLO modellerine nazaran daha büyük boyutta olup (Resim 3.2.7) YOLOv5x modelidir. YOLOv5x'in YOLOv5s'e göre daha fazla nesne tespiti sağladığı görülmektedir (Resim 3.2.8). Bunu nedeni olarak doğrudan konvolüsyonel sinir ağıları sayısı orandır.

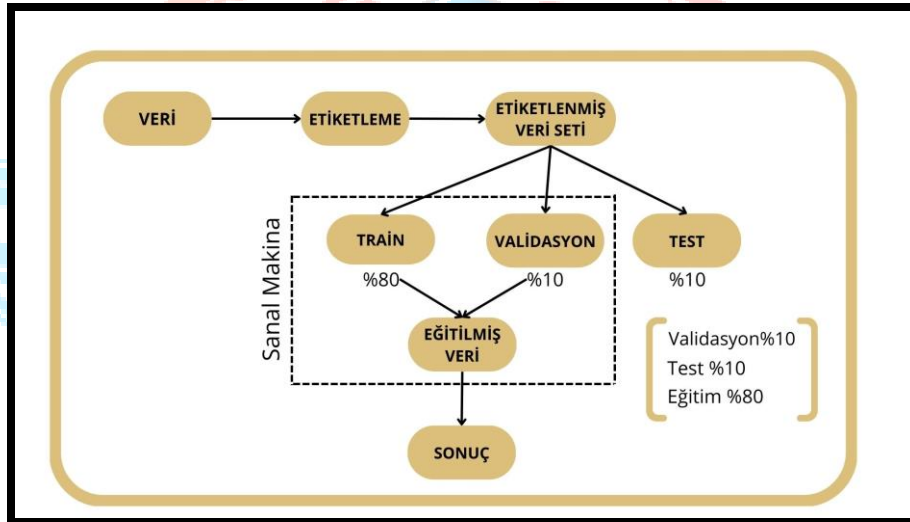


Resim 3.2.7

				
Nano YOLOv5n	Small YOLOv5s	Medium YOLOv5m	Large YOLOv5l	XLarge YOLOv5x
4 MB _{FP16} 6.3 ms _{V100} 28.4 mAP _{COCO}	14 MB _{FP16} 6.4 ms _{V100} 37.2 mAP _{COCO}	41 MB _{FP16} 8.2 ms _{V100} 45.2 mAP _{COCO}	89 MB _{FP16} 10.1 ms _{V100} 48.8 mAP _{COCO}	166 MB _{FP16} 12.1 ms _{V100} 50.7 mAP _{COCO}

Resim 3.2.8

Bir grafik tasarım üzerinde anlatmak gerekirse sistemimiz aşağıdaki gibi çalışmaktadır (Resim 3.2.9)

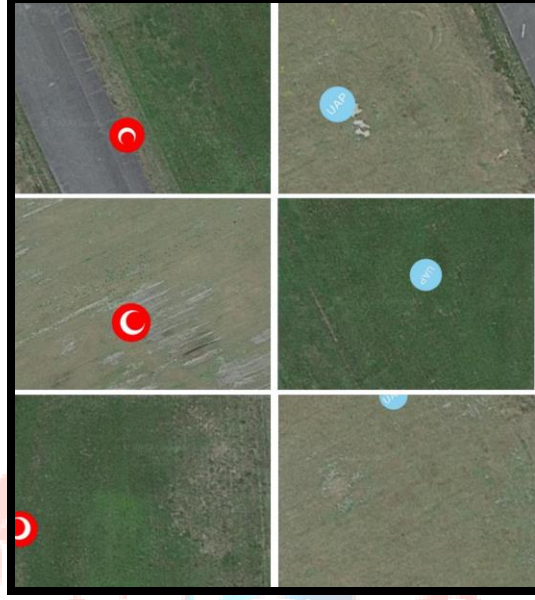


Resim 3.2.9

4. Özgünlük

Takımımız özgünlük çalışmalarını veri tabanı kısmı üzerinde gerçekleştirmiştir. Algoritmamızın eğitilmesi için gerekli miktarlarda UAP ve UAİ görsellerinin bulunmaması ön kabulünden yola çıkılarak yapay UAP ve UAİ görselleri oluşturan bir sistemin geliştirilmesine karar kılınmıştır.

Bunun için Ön Tasarım Raporu'nda da belirtildiği üzere ilk başta Python programlama dilinin OpenCV kütüphanesi kullanılarak yapay görseller oluşturulmuştur (Resim 4.1). Lakin, rapor sonuçlarında hakem heyetinden gelen geri bildirimler doğrultusunda bu görsellerin gerekli niteliklerde olmadığına karar kılınmıştır.



Resim 4.1

Yapay görsellerin daha gerçekçi olması ve üç boyuta sahip olması istenildiğinden geliştirilmelerin bir oyun motoru üzerinde yapılmasına karar verilmiştir. Oyun motoru olarak Rockstar markasının geliştirdiği “Rockstar Advanced Game Engine” kullanılmıştır.

Yapay görseller aşağıdaki süreç ile üretilmiştir;

1. Adobe Photoshop programı kullanılarak UAP ve UAI nesneleri için 1024x1024 boyutlarında örnek görseller oluşturulmuştur.
2. Nesnelerin her zaman düzgün bir görüntüde ve açıda görüntüleri elde edilemeyeceği için Adobe Photoshop programı yardımıyla kırışıklık efekti eklenmiştir.
3. Oluşturulan bu görseller Autodesk'in 3ds Max programı ve oyun motoru yardımcı eklentisi (GIMS Evo) ile oyun içi 3D modellere dönüştürülmüştür.
4. Oyun modlama yardımcısı OpenIV ile oluşturulan modeller oyuna entegre edilmiştir.
5. Başka bir oyun içi modu ile (Menyoo PC) oluşturulan modeller, oyun içinde farklı yerlere konularak farklı açı ve durumlardan görseller alınmıştır.
6. Böylece UAP ve UAI'nin başta üzerinde bir nesnenin olması gibi durumlar simüle edilmiştir.

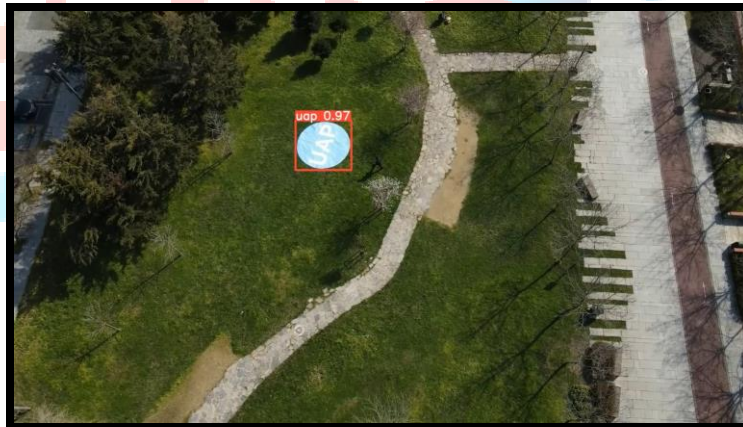
Sistemimizin çıktısı olan bazı görseller aşağıdaki gibidir (Resim 4.2):



Resim 4.2

5. Sonuçlar ve İnceleme

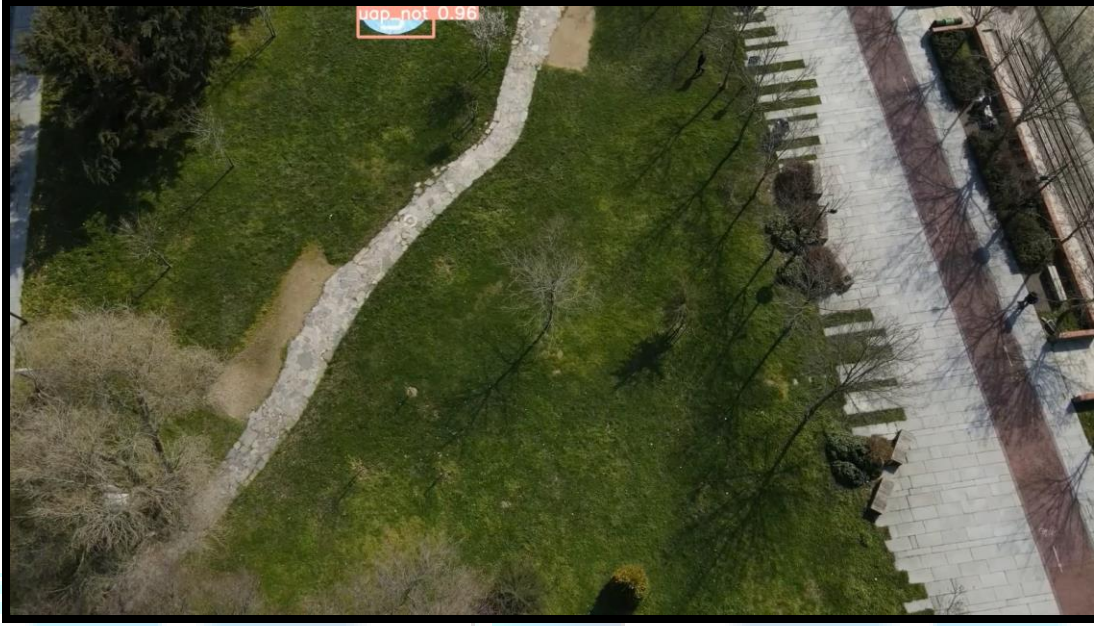
Sistemimiz özellikle yapay görsellerin kullanılmasının ardından iniş durum kontrolünü istenilen niteliklerde yapabilmektedir.



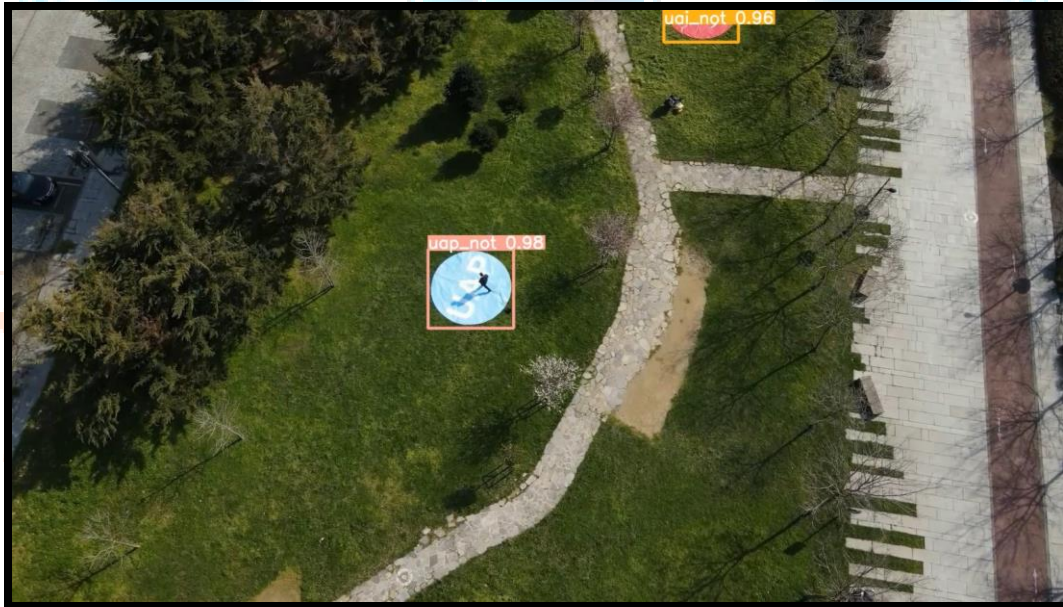
Resim 5.1

Yukarıdaki Resim 5.2'de de görüleceği üzere sistemimiz UAP'nin tüm alanının görüş alanımızın içerisinde olup olamamasını ve üzerinde yabancı bir cismin olup olamamasını kontrol ederek iniş izni vermektedir.

Aksi olan durumlarda (Resim 5.2 ve Resim 5.3) sistemimiz nesneyi “UAP_NOT veya UAI_NOT” olarak nitelendirmektedir. Böylece iniş durumları kontrol edilebilmektedir.

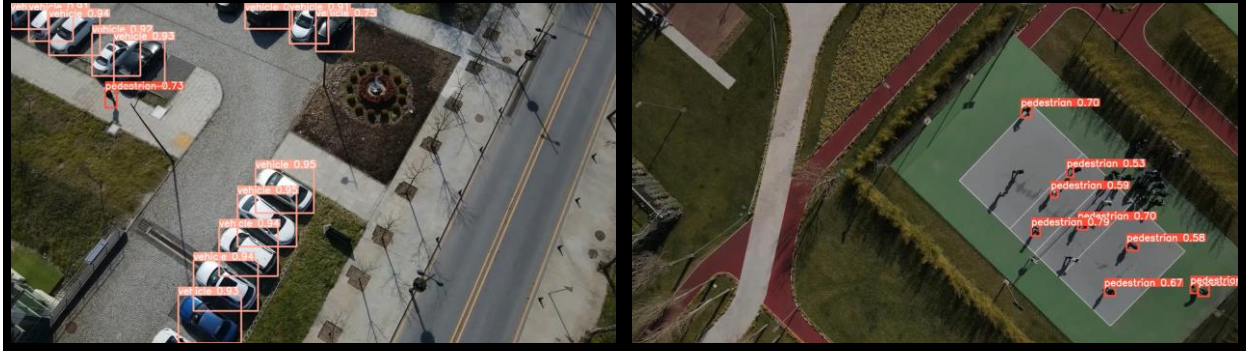


Resim 5.2 – UAP’nin tüm alanının görüş alanında olmaması durumu



Resim 5.3 – UAP veya UAI’nin üzerinde yabancı cisim olma durumu

İniş durum kontrolünün yanı sıra nesne tespitinde de sistemimiz iyi bir performans göstermektedir.



Resim 5.4 ve Resim 5.5

Resim 5.4 ve Resim 5.5'ten de anlaşılacağı üzere sistemimiz araç tespitleri noktasında problem yaşamamaktadır ancak yaya tespitleri konusunda hala istenilen orana erişilememiştir. Tespit edilen nesnelerin sınıflandırılması ve hedeflenen tespit oranına erişilebilmesi için takımımız çalışmalarını sürdürmektedir.

6. Kaynakça

- [1] Ganhi, R. (2018) R-CNN, Fast R-CNN, Faster R-CNN, YOLO — Object Detection Algorithms. Towards Data Science, Medium
- [2] Nelson, J. & Solawetz, J. (2020). YOLOv5 is Here. Roboflow
- [3] Nelson, J. & Solawetz, J. (2020). Responding the Controversy about YOLOv5. Roboflow
- [4] Nepal, U., & Eslamiat, H. (2022). Comparing YOLOv3, YOLOv4 and YOLOv5 for Autonomous Landing Spot Detection in Faulty UAVs. Sensors, 22(2), 464. MDPI AG.
- [5] Rath, S.R. Object Detection using PyTorch YOLOv5. Debugger CAFE
- [6] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (SS. 779-788).
- [7] Zambare, S. Object detection: using non-max suppression over YOLOv2. Medium